

Езици за функционално програмиране - конспект

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуална машина. Разпределение на паметта, начин на извод.

Тема 3. Организация на кода. Модули и функции – дефиниции, атрибути, предаване на параметри, видимост.

Тема 4. Система от типове данни – единични и комбинирани. S-изрази. Функции от първи клас. Анонимни функции.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции.

Тема 7. Условни изрази – case, if.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Тема 10. Полиморфизъм.

Тема 11. Работа с протоколи за обмен на данни. Вградени протоколи.

Тема 12. Конкурентност в BEAM виртуалната машина. Създаване и управление на процеси. Работа със съобщения.

Тема 13. Процеси от страна на сървъра. Управление на състоянието на процеса.

Тема 14. Основен процес на сървъра – генериране и управление.

дисциплина: Езици за функционално програмиране

лектор: гл. ас. д-р инж. Димитричка Николаева

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

Съдържание:

1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици.

- 1.1. Императивни езици.
- 1.2. Дескриптивни езици.
- 1.3. Характеристики на дескриптивните езици за програмиране.
- 1.4. Основи на функционалното програмиране.
 - 1.4.1. Обща характеристика на функционалния стил на програмиране.
 - 1.4.2. Теоретичният модел на функционалното програмиране.
 - 1.4.3. Особенности на ФП.
 - 1.4.4. Дефиниране на ФП.
 - 1.4.5. Класификация на езици за функционално програмиране.
- 1.5. Характерни особености на функционалния стил на програмиране. Основни компоненти на функционалните програми.

2. Основни принципи на функционалното програмиране, споделяне на добри практики и програмни парадигми.

- 2.1. Основни принципи на функционалното програмиране.
 - 2.1.1. Неизменяемост.
 - 2.1.2. Дисциплинирано състояние.

- 2.1.3. Чисти функции и без странични ефекти.
- 2.1.4. Първокласни функции и функции от по-висок ред.
- 2.1.5. Типови системи.
- 2.1.6. Референтна прозрачност.
- 2.2. Споделяне на добри практики.
- 2.3. Програмни парадигми.
 - 2.3.1. Императивна парадигма.
 - 2.3.2. Процедурна парадигма.
 - 2.3.3. Обектно-ориентирана парадигма.
 - 2.3.4. Декларативна парадигма.

3. Езици за функционално програмиране (ФП).

- 3.1. История на функционалното програмиране.
- 3.2. ЛИСП
- 3.3. Metalanguage
- 3.4. Hope
- 3.5. Haskell
- 3.6. Clojure
- 3.7. Elm
- 3.8. Elixir
- 3.9. F#
- 3.10. Idris
- 3.11. JavaScript
- 3.12. Kotlin
- 3.13. Scala

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици.

Императивно програмиране (ИП) (imperare от латински нареждам, заповядвам). **ИП е парадигма** в програмирането, при която една програма се състои от **редица команди, които определят какво и в какъв ред компютърът трябва да направи.**
[1]

Дескриптивно програмиране (ДП) е парадигма в програмирането, при която в програмата се **описват свойствата на желания резултат, но не се показва пътя как да бъде постигнато това.**

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

1.1. Императивни езици (ИЕ)

- езици от високо ниво: ПАСКАЛ, АДА, С и подобните на тях;
- **ИЕ** указват как да се получи даден резултат във вид на фиксирана изчислителна процедура. Тя се определя чрез последователност от оператори, като е важна тяхната подредба;
- **ИЕ** използват оператор за присвояване;
- **ИЕ** зависят от архитектурата на компютрите, като за разлика от асемблерните езици, операторите им представляват обобщени групи от машинни команди.

1.2. Дескриптивни езици (ДЕ)

- езици за функционално програмиране (апликативни езици) и езици за логическо програмиране;
- явно се указва какви свойства има желаният резултат, без да се описва точно как се получава този резултат.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

Таблица 1. Сравнителна характеристика между императивен и дескриптивен език

Е	Схема	Предимство	Проблеми	Програмни езици
Императивен език	ПРОГРАМА = АЛГОРИТЪМ + СТРУКТУРИ ОТ ДАННИ	широко разпространение на компютри с Нойманова архитектура.	- относително малка мощност; - ниска производителност; - липса на гъвкавост; - не притежават полезни математически свойства.	машинните, асемблерните езици и FORTRAN, BASIC, PL/I, ALGOL, PASCAL, ADA и Cobol, C, асемблерните езици и други.
Дескриптивен език	ПРОГРАМА = СПИСЪК ОТ ФУНКЦИИ ИЛИ РАВЕНСТВА ИЛИ ПРАВИЛА + ФАКТИ (ИЗПЪЛНЕНИЕ = РЕДУКЦИЯ ИЛИ ИЗВОД (ДОКАЗАТЕЛСТВО))	налага разработване на нови ненойманови архитектури наречени под общото име пето поколение компютри	не може да се направи ефективна реализация на съществуващите Нойманови компютри	LISP, PROLOG, POP- 11, TABLOG и други

1.3. Характеристики на дескриптивните езици за програмиране

- работа със списъци;
- дефиниране на рекурсивни функции за нечислови (символни) пресмятания;
- динамично формиране на обръщения към функции;
- използване на агрегатни функции;
- съпоставяне по образец;
- генериране на алтернативни решения;
- реализиране на подхода “проба и грешка” или обхождане с връщане назад;
- въвеждане на знания във вид на формули (логически);
- използване на логически извод;
- визуален и звуков вход и изход.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

Защо функционално програмиране?

- кратки и ясни програми (изразителност);
- лесна проверка за коректност;
- при еднакви входни данни връщат един и същ резултат (референциална прозрачност);

което позволява...

- избягване на повторно пресмятане на резултати чрез запомняне (мемоизация);
- премахване на части от програмата, които не участват в крайния резултат (мъртъв код);
- пренареждане на програмата за по-ефективно изпълнение (стратегия за оценяване);
- паралелно изпълнение на независими части от програмата (паралелизация).

Във функционалното програмиране има функции с параметри (абстракция), които могат:

- да се прилагат над аргументи, (апликация);
- да са други функции (функции от висок ред);
- и да се дефинират чрез себе си, (рекурсия);

...но ФП няма :

- памет;
- присвояване;
- цикли;
- прескачане (goto, break, return).

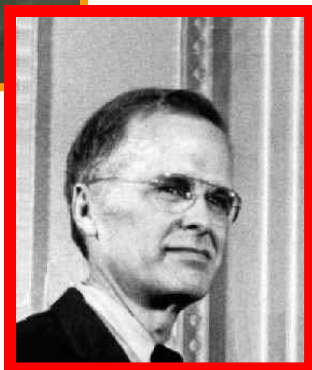
Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

1.4. Основи на функционалното програмиране

1.4.1. Обща характеристика на функционалния стил на програмиране

Програмирането във функционален стил се състои от: дефиниране на функции, които пресмятат (връщат) стойности. При това тези стойности еднозначно се определят от стойностите на съответните аргументи (фактически параметри); прилагане (апликация) на тези функции върху подходящи аргументи, които също могат да бъдат обръщения към функции, тъй като всяка функция връща стойност.

1.4.2. Теоретичен модел на функционалното програмиране



- λ -изчисление въведено от Алонсо Чърч (Alonzo Church);
- математически функции и λ -смятане;
- метод за дефиниране на функции без име - Lambda функции (Alonzo Church, 1941);
- Джон Бекус през 1978 поставя началото на функционалния стил за програмиране, използващ примитивни форми, дефиниции или функционални форми.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

1.4.3. Особености на ФП

- Програмата представлява съвкупност от дефиниции на функции;
- Възможно е многократно влягане на функциите една в друга;
- Единствения начин за разделяне на програмата на части е въвеждането на име на функция и задаването на това име на израз, който пресмята стойността на функцията програмите представляват дефиниране на функции;
- Единственото правило за композиция е суперпозицията на функции;
- Не използват променливи и оператори за присвояване – итеративните конструкции не са възможни;
- Повторенията се реализират чрез рекурсия;
- Еднакво представяне на код (функции) и данни;
- Изпълнението на функция винаги дава същия резултат при същите входни параметри;
- Проста семантика;
- Задават множества от примитивни функции и функционални форми за конструиране на съставни функции, операции, и някои структури за представяне на данните;
- Използват се по-често интерпретатори.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

1.4.4. Дефиниране на ФП

Защо се нарича функционално програмиране?

ФП получава името си от подход към изграждане на софтуер, създаден въз основа на чисти функции. Основата на всички **ФЕ** за програмиране е Lambda Calculus (λ -смятане), това е „най-малкият универсален език за програмиране в света“.

ФП се основава на математически функции, които използват условни изрази и рекурсия за извършване на изчисленията. **ФП** се **характеризира с**:

- данни, които са неизменни;
- избягват се странични ефекти;
- по-лесна поддръжка;
- референтна прозрачност;
- това, че модулният дизайн позволява малките модули, да се кодират бързо и да се преизползват, което намалява времето за разработка на програмата. Модулите могат да бъдат тествани поотделно, което намалява времето за отстраняване на грешки.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

1.4.5. Класификация на езици за функционално програмиране

- **Чист (строг) функционален език**

Управляващите конструкции са функциите, които не допускат странични ефекти.

Пример: ЧИСТ LISP, Haskell, HOPE, Miranda, FP, W и други;

- **Нечист (нестрог) функционален език**

Управляващите конструкции са функции и императивни конструкции, като функциите допускат странични ефекти.

Пример: Scheme (Ским), Common Lisp, Metalanguage (1980), Golden Lisp (диалекти на LISP), APL;

- **Логически и функционални езици**

Пример: N, F, R, HASL, FUNLOG, QUTE, LOGLISP, QLOG, PIL, Prolog-with-Equality, EQLOG, TABLOG, UNICORN, Pop-11, LEAF, APPLOG.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

Функционален стил на програмиране

- дефиниране на функции, които пресмятат (връщат) стойности, които еднозначно се определят от стойностите на съответните аргументи (фактически параметри);
- прилагане (апликация) на тези функции върху подходящи аргументи, които също могат да бъдат обръщения към функции.

предимства

- лесна проверка и поправка на съответните програми;
- подходящ при проектирането на езици за програмиране, предназначени за многопроцесорни компютри, в които много от пресмятанията се извършват паралелно;
- могат да бъдат доказвани точни свойства на функционалните програми.

недостатъци

- многократно пресмятане на едни и същи изрази;
- неестествено и неефективно е използването им при решаване на задачи от процедурен характер.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

Чист подход

В чисто функционалните езици параметрите и аргументите не могат да се променят при изпълнението на дадена функция. Т.е не се използва оператор за присвояване и единствената стойност се получава като функционална стойност. При обръщение към дадена функция първо се пресмятат или оценяват аргументите, след което се прилага функцията, такава каквато е дефинирана върху оценените аргументи, и се получава стойността на функцията за дадените аргументи. Тъй като функционалните езици работят със списъци, то резултатът може да е списък от резултати, напомнящ векторна стойност.

Пример за програмни езици, които използват чист подход:

Haskell, Lisp, Racket, Erlang и други.

Нечист подход

В някои езици, не може да се предаде функция на друга функция. Например, Python, е проектиран така, че да поддържа множество парадигми за програмиране, което го прави гъвкав.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

2. Основни принципи на функционалното програмиране, споделяне на добри практики и програмни парадигми

2.1. Основни принципи на функционалното програмиране

- Immutability (Неизменяемост)
- Disciplined state (Дисциплинирано състояние)
- Pure functions and no side effects/disciplined states (Чисти функции и без странични ефекти/дисциплинирани състояния)
- First class functions and high order functions (Първокласни функции и функции от висок ред)
- Type systems (Типови системи)
- Referential transparency (Референтна прозрачност)

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

2.1.1. Неизменяемост

Концепцията за неизменност е ЯДРОТО на **ФП** и това означава, че след като присвоите стойност на нещо, тази стойност няма да се промени.

2.1.2. Дисциплинирано състояние

Споделеното променящо се състояние не е желателно, защото е много по-трудно да се мащабира и да се изпълнява едновременно. Този аспект вече се среща в корпоративни езици като Java, .NET, Ruby и Python.

2.1.3. Чисти функции и без странични ефекти

Чистите функции са тези без странични ефекти. Страничните ефекти са лоши, тъй като са непредсказуеми и затрудняват тестването на софтуера.

2.1.4. Първокласни функции и функции от по-висок ред

- **Първокласен** означава, че езикът третира функциите като първокласни или с други думи, това означава да имате езикова поддръжка за предаване на функции като аргументи на други функции и за връщане на стойности като функции. *Първокласната функция също така предполага, че езикът позволява да се съхраняват функции, като променливи или всяка друга структура от данни.*
- **Функциите от по-висок ред** са математическо понятие, при което функциите оперират с други функции. **Функциите от по-висок ред** са свързани с първокласни функции. Функциите от по-висок ред често означават езикова поддръжка за частично функционално приложение и Currying.
- **Частични функции** са тези функции при които може да се фиксира стойност (аргумент) към определена функция, която по-късно може или не може да бъде променена .
- **Къриране** е техника за трансформиране на функция с множество параметри в последователност от функции, като всяка функция има един аргумент. Къриране, почти винаги използва езици като: **Metalanguage** и **Haskell**, докато езикът Scala не прилага тази техника.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

2.1.5. Типови системи

Типовата системата е свързана с компилатора. Идеята е проста: създаване на типова система и по този начин използване на компилатора, за предотвратяване на каквито и да било грешки.

2.1.6. Референтна прозрачност

При кодирате на програма с истински функции, данните в нея остават **неизменни и говорим за Референтната прозрачност**. Това означава, че един и същ вход винаги ще връща един и същ изход. Тази концепция или принцип се използва най-вече при замяна на същите изходи с **константа**, за да се намали времето и пространството, необходими за програмата.

2.2. Споделяне на добри практики за функционално програмиране

- Желателно е да се използват възможно най-**много истински функции**. Но не трябва да поддържате всички функции във вашата програма чисто верни.
- Проверявайте дали данните, които използвате във вашите функции, имат **локален обхват** и са **неизменни**.
- Състоянието на данните в програмата трябва да бъде **дисциплинирано**. Не трябва да го споделяте между множество функции.
- Използвайте **функции от висок ред** като **първокласни функции** в програма си.
- Замяна на същите изходи на същите входове с **константа** като **референтна прозрачност**.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

2.3. Програмни парадигми

2.3.1. Императивна парадигма

Императивното програмиране използва подход стъпка по стъпка за изпълнение на задача. Разработчикът предоставя команди, които описват точно как да се изпълни задачата от началото до края. По време на процеса на изпълнение на командите, кодът също променя състоянието на приложението, което включва данните за приложението.

2.3.2. Процедурна парадигма

Процедурното програмиране реализира императивно програмиране, но добавя функционалност като кодови блокове и процедури за разбиване на кода. Компиляторът или интерпретаторът все още произвежда машинен код, който се изпълнява стъпка по стъпка, но използването на процедури улеснява разработчика да следва кода и да разбере как работи той. Примери за езици, които прилагат процедурната парадигмата са C и Pascal. Ранните езици, като Basic.

2.3.3. Обектно-ориентирана парадигма (ООП)

ООП използва концепция за обекти, за да скрие кода, но по-важно, за да направи моделирането за реалния свят по-лесно. Разработчик създава кодови обекти, които имитират обекти от реалния свят. Тези обекти включват свойства, методи и събития. Примери за езици, които прилагат обектно-ориентираната парадигма са C++ и Java. Езиците, които прилагат обектно-ориентираните парадигми, също изпълняват, както процедурната така и императивната парадигма.

2.3.4. Декларативна парадигма

Функционалното програмиране всъщност изпълнява парадигмата за декларативно програмиране, но двете парадигми са отделни. Други парадигми, като логическо програмиране, реализирано от езика Prolog, също поддържат декларативната програмна парадигма. Краткият изглед на декларативното програмиране е, че то прави следното:

- Описва какво трябва да прави кодът, а не как да го направите
- Определя функции, които са сравнително прозрачни (без странични ефекти)
- Осигурява ясно съответствие с математическата логика.

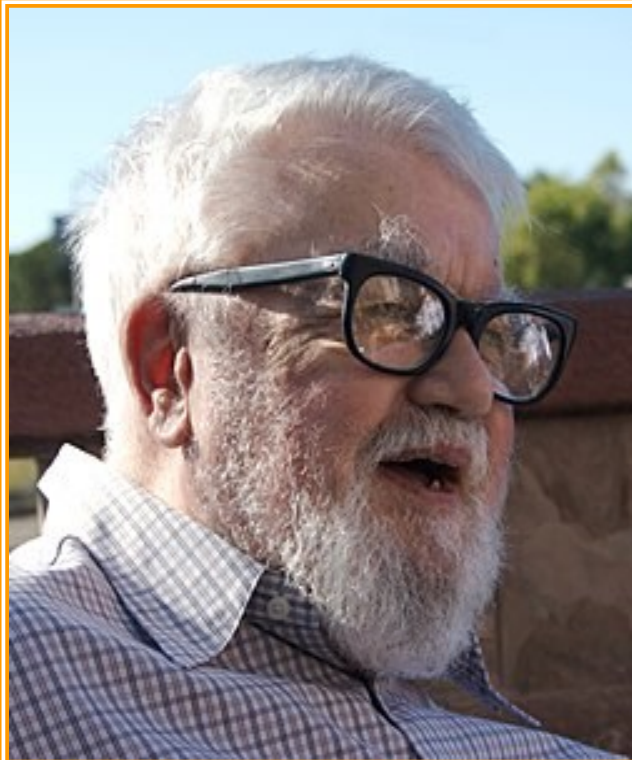
Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3. Езици за функционално програмиране (ФП).

3.1. История на функционалното програмиране

- (1936) Church и Rosser дефинират λ -смятането
- (1960) McCarthy създава първия функционален език LISP
- (1975) Steele и Sussman създават Scheme, диалект на LISP
- (1977) Backus (авторът на FORTRAN) популяризира функционалния стил
- (1985) Turner създава Miranda, първият комерсиален чист функционален език
- (1990) Публикувана е първата версия на Haskell
- (1998) Отваряне на кода на реализацията на Erlang
- (1990–2000) Функционални елементи в императивни езици: Python (1991), JavaScript (1995), Ruby (1995), ActionScript (1998)
- (от 2000 до днес) Функционалният стил на програмиране превзема света:
 - ❑ Scala (2003);
 - ❑ F# (2005);
 - ❑ C# (2007);
 - ❑ Clojure (2007);
 - ❑ C++11 (2011);
 - ❑ Elixir (2011);
 - ❑ Java 8 (2014).

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).



3.2. ЛИСП

През 1958-1961г. Джон Маккарти разработва езика за функционално програмиране LISP - LISt Processing languages или „език за обработка на списъци“. Първо той е описан като математически формализъм и не е бил предназначен за програмиране на задачи от изкуствен интелект. От него произлизат множество диалекти, най-известните сред които са Scheme и Common Lisp. Той води до развитие на редица програмни езици, като: Elixir, Haskell, JavaScript, Perl, Python, Ruby, Scala, Swift, Smalltalk и други.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

ЛИСП се характеризира

- като най-старият и най-разпространеният;
- език с много диалекти;
- език, чийто първи диалект е за IBM 704;
- като език за символна обработка и задачи от ИИ;
- като безтипов език;
- език, който притежава императивни черти – променливи, присвояване и итерация;
- език с префиксен запис - произволен брой аргументи (funname arg1 arg2 ...argn);
- език, чийто механизъм на работа е: четене – оценка – отпечатване.

Предимства на ЛИСП

- прост синтаксис;
- поддържа мощна модулна декомпозиция на програмите;
- могат да се правят процедурни абстракции, абстракции на данни;
- могат да се използват процедури от по-висок ред за обхващане на общите неща при прилагане на процедури;
- могат да се моделират локални състояния с присвояване и промяна в данните;
- могат да се свързват части от програмите с отложено оценяване.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3.3. Metalanguage

Близо две десетилетия по-късно следващият голям напредък се случва в Университета в Единбург, където изследователите създават **Metalanguage**, езикът който да работи за тяхната автоматична система за доказване на теореми (LCF). Metalanguage покрива изискванията и през 1978 г. бива установено, че той може да бъде полезен като общ език за програмиране. Версиите включват Standard Metalanguage на Ню Джърси и CAML.

3.4. Hope

През 70-те години на миналия век в университета в Единбург е разработен **Hope** (Надеждата). Той е малък функционален език за програмиране, който предшества Миранда и Хаскел и се появява едновременно с Metalanguage. **Hope** е получен от NPL - прост функционален език за програмиране със съвпадение на шаблони, разработен от Род Бърстал и Джон Дарлингтън през 1977 г. Езикът позволява определени набори и логически конструкции да се появяват от дясната страна на дефинициите.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3.5. Haskell

През 1990 г. е пусната първата версия на **Haskell** (Хаскъл). Функционален език за програмиране, продукт на комисия, която започва разработването му през 1987 г. Носи името на Хаскел Къри, творещ в областта на математическата логика. Неговото лого е Ламбда, тъй като е базиран на ламбда смятане. **Haskell** в частност е полиморфично статично-типизиран, „мързелив“, чисто функционален език, доста различен в сравнение с другите програмни езици.

3.6. Clojure

Clojure е диалект на Lisp, създаден от Rich Hickey. За разлика от другите диалекти на LISP, Clojure работи на платформата Java и е компилиран в байт код JVM. Предимството, което предлага е интеграцията му с Java. Hickey добавя, че **Clojure** осигурява лесен достъп до рамките на Java, с незадължителни подсказки за тип и заключение за тип, за да се гарантира корекността им.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3.7. Elm

Elm (Бряст) е името на чисто функционален език, проектиран за първи път от Evan Czaplicki (Еван Чаплички) през 2012 г. Той е популярен сред веб разработчиците, които го харесват за изграждане на потребителски интерфейси. Компиляторът на Elm е насочен към HTML, CSS и JavaScript. Работи с JavaScript библиотеки по същия начин, както Clojure работи с Java библиотеки. Различното при Elm е, че той използва статична проверка на типа, която подпомага елиминирането на изключенията по време на изпълнение, тъй като грешките ще бъдат засечени по време на компилиране. Липсата на видими грешки в потребителския край е очевидно предимство за езика.

3.8. Elixir

Според описанието поместено в сайта на Elixir: „Езикът е динамичен, функционален, предназначен за изграждане на мащабируеми и поддържащи се приложения.“ Еликсир е насочен към създаване на системи с висока достъпност и ниско закъснение. Той работи на Erlang VM, система, която се използва от Heroku, WhatsApp, Klarna и други за „разпространени, несъвместими приложения“. Предимството за програмистите на Elixir е, че те могат да се позовават на функция Erlang, без да правят разходи за изпълнение.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3.9. JavaScript

Въпреки че JavaScript не е строго функционален език за програмиране, тъй като позволява и обектно-ориентиран подход, той има компонент, който произтича от функционалното програмиране. Това включва функции от по-висок ред. Също така по-новите версии на стандарта ECMAScript предоставят поправки на проблема с изменяемостта.

3.10. Kotlin

Сред най-новите езици (пуснати едва през февруари 2016 г.) е Kotlin, който в продължение на пет години е разработван и тестван. Компании като Amex, NBC Digital, Expedia и Gradle го използват. Той не е чист функционален език. Подобно на JavaScript, Kotlin е способен както на обектно-ориентирани, така и на функционални конструкции. Освен това е напълно оперативно съвместим с езиците за програмиране като Java. Сред предимствата, които предлага е, че програмирането е по-сбито от програмирането на Java.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3.11. F#

F# е отворен код, крос-платформа, функционално-първи език за програмиране. Всеки може да се присъедини към F# Software Foundation. Сайтът на фондацията обобщава езика по този начин: F# работи на Linux, Mac OS X, Android, iOS, Windows, GPU и браузъри. Той е свободен за използване и е с отворен код под лиценз, одобрен от OSI. F# се използва в широк спектър от области на приложение и се поддържа както от активна отворена общност, така и от водещи в индустрията компании, предоставящи професионални инструменти.

3.12. Idris (Идрис)

Idris е чист ФЕ за програмиране с общо предназначение, който е повлиян от Haskell и Metalanguage. Той се характеризира със зависими типове, което дава възможност „типовете да се определят на стойности, което означава, че някои аспекти от поведението на програмата могат да бъдат конкретизирани точно в типа.“ Той се отличава и чрез компилиране с нетърпелива оценка.

Тема 1. Същност на „утвърдителното“ програмиране. Дескриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране (ФП).

3.13. Scala

Scala е друг хибрид, тъй като предлага както обектно-ориентирано, така и функционално програмиране. „Статичните типове на Scala помагат да се избегнат грешки в сложните приложения, а JVM и JavaScript по време на работа позволяват да се изградят високоефективни системи с лесен достъп до огромни системи от библиотеки“.

„Един мит е, че функционалното програмиране е трудно или твърде академично. Това е просто още едно умение за учене. Необходимо е време и практика, за да се овладее, нещо различно от пътуването, през което разработчиците преминаха, за да научат обектно-ориентирано програмиране в края на 80-те / началото на 90-те. Технологиите продължават да се развиват. Това е една от причините да обичам софтуера. Да се научиш как да учиш е толкова важно, колкото и това, което знаеш.“

Telstra GM Mark Moloney

Използвана литература:

- [1] [Simon St. Laurent, J. David Eisenberg](#), „Introducing Elixir: Getting Started in Functional Programming 2nd Edition”, O'Reilly Media; 2nd edition (January 24, 2017), ISBN-13: 978-1491956779, ISBN-10: 1491956771
- [2] <http://www.referati.org/uvod-vyv-funkcionalnoto-programirane/39464/ref>
- [3] <https://bg.continuousdev.com/functional-programming-languages-past-present-and-future-2-34003-12640>
- [4] Functional Programming For Dummies, Copyright © 2019 by John Wiley & Sons, Inc., Hoboken, New Jersey
- [5] <https://bg.continuousdev.com/functional-programming-languages-past-present-and-future-2-34003-12640>
- [6] https://subscription.packtpub.com/book/application_development/9781786461483/1/ch01lvl1sec8/principles-of-functional-programming
- [7] <https://techmused.com/what-is-functional-programming-and-its-basic-principles/>
<https://translate.google.com/translate?sl=en&tl=bg&u=https://www.lucidchart.com/techblog/2017/11/29/functional-programming-principles-every-imperative-programmer-should-use/>